

PRODUCT REQUIREMENTS DOCUMENT

Emergency Response Voice Agent

Real-time, voice-first emergency reporting and coordination platform

Version	1.0
Status	Product Definition
Target	AssemblyAI Voice Agent Hackathon
Platform	Web application + backend services
Primary Interface	Real-time voice
Core Technology	AssemblyAI Voice AI

Core principle: When people are under stress, they should not have to learn how to use an emergency application. They should be able to simply talk.

1. Executive Summary

The Emergency Response Voice Agent is a real-time, voice-first AI system designed to help people report and manage emergency situations when traditional smartphone interfaces are too slow, difficult, or unsafe to use.

Instead of requiring users to open an application, fill out forms, identify an emergency category, and manually enter information, the user simply speaks naturally. The agent listens, understands the situation, extracts critical information, asks context-aware follow-up questions, determines severity, provides appropriate non-diagnostic safety guidance, and creates a structured emergency incident.

A second interface provides an Emergency Command Center, allowing an operator to monitor incidents, investigate reports, communicate with users/responders, and control supported workflows using voice.

Supported scenarios: road accidents, medical emergencies, fires, gas/electrical hazards, natural disasters, industrial incidents, campus/public-safety incidents, and other emergency reports.

2. Problem Statement

- Panic and cognitive overload during emergencies.
- Injuries or physical limitations that make typing difficult.
- Poor ability to navigate smartphone interfaces while under pressure.
- Lack of knowledge about what information emergency services need.
- Incomplete or unstructured incident reports.
- Repeated questioning by different responders.
- Delayed communication between witnesses and emergency teams.

Conventional workflow: Open app -> login -> choose emergency -> enter location -> fill form -> submit. The proposed workflow is: User speaks -> AI understands -> AI asks relevant questions -> AI structures information -> AI assesses urgency -> AI initiates the appropriate workflow -> user receives guidance.

3. Product Vision

Create a voice-native emergency coordination layer connecting citizens, AI, responders, and emergency operations centers. The long-term vision is a conversational emergency interface that reduces friction between an incident and the people responsible for responding to it.

4. Goals

- G1 - Hands-free emergency reporting. Users should be able to report an emergency through voice.
- G2 - Speech-to-structure. Convert unstructured speech into a structured incident.
- G3 - Dynamic questioning. Ask only the most important missing questions.
- G4 - Safety-oriented guidance. Provide conservative guidance without diagnosis or unsupported certainty.
- G5 - Command center. Give operators a live view of active incidents.
- G6 - Agentic behavior. Use explicit tools to perform controlled actions.
- G7 - AssemblyAI showcase. Demonstrate realtime speech recognition, context, turn detection, interruptions, and voice responses.

5. Non-Goals

- Replacing emergency dispatchers or trained professionals.
- Providing medical diagnosis.
- Autonomously deploying emergency vehicles.

- Making final medical, legal, or law-enforcement decisions.
- Automatically contacting real emergency services during the hackathon unless an authorized integration exists.
- Building nationwide emergency infrastructure.

6. Target Users

Primary - Emergency Reporter: driver, pedestrian, student, worker, farmer, resident, tourist, or bystander.

Secondary - Emergency Operator: campus security operator, industrial safety officer, event security operator, or emergency coordination staff.

Tertiary - Responder: first responder, security officer, fire/medical response team, or industrial safety team.

7. Core Use Cases

UC-01	Road Accident	Gather accident type, location, vehicles, people affected, injuries, trapped people, fire/smoke, and traffic obstruction.
UC-02	Medical Emergency	Gather consciousness, breathing status, visible bleeding, approximate age, location, and number of people affected; provide conservative guidance and escalation.
UC-03	Fire	Determine building type, location, people inside, evacuation feasibility, smoke/fire visibility, and hazards.
UC-04	Gas / Chemical Leak	Identify location, possible source, exposure, symptoms, evacuation status, and safety risks.
UC-05	Natural Disaster	Support flood, earthquake, cyclone, landslide, and storm reports using location, people, vulnerabilities, danger, and evacuation information.
UC-06	Industrial Emergency	Use site manuals, equipment information, safety procedures, and site maps to support workers.
UC-07	Public Safety Incident	Create a structured incident for events such as an unconscious person or crowd hazard.
UC-08	Multiple Witnesses	Fuse reports describing the same event into one incident using time, location, type, and description similarity.

8. Core Product Architecture

User voice -> AssemblyAI voice layer -> Agent runtime -> RAG / tools / database -> Incident engine -> Citizen voice response + Command Center

9. Major Components

9.1 Voice Interface. Microphone access, realtime audio streaming, voice activity detection, conversation state, spoken responses, and interruption handling.

9.2 AssemblyAI Integration. Realtime speech-to-text, turn detection, voice activity detection, interruption handling, and context-aware transcription.

9.3 Conversation Engine. Maintains conversation, incident, user, location, risk, and confidence state.

9.4 Dynamic Question Engine. Selects the most important missing information instead of running a fixed questionnaire.

9.5 Emergency Classification. Classifies incident type and severity with an internal confidence score.

9.6 Incident Extraction. Continuously extracts structured facts from the conversation.

9.7 Location System. Supports browser/device geolocation and user-described locations; distinguishes exact, approximate, described, and unknown locations.

9.8 Incident State Machine. Tracks the lifecycle from reported through verification, triage, response, resolution, and closure.

9.9 Duplicate Detection. Finds potentially duplicate incidents using time, geographic proximity, type, and description similarity.

9.10 Command Center. Live incident map/list, filters, detail pages, transcripts, timelines, extracted facts, responder status, and controlled actions.

9.11 Voice-Controlled Command Center. Operators can query incidents and execute supported actions by voice.

9.12 Safety/RAG Layer. Retrieves trusted emergency procedures where appropriate and applies safety validation before responding.

10. Incident State Machine

REPORTED -> VERIFYING -> TRIAGED -> DISPATCH_PENDING -> RESPONDER_ASSIGNED -> IN_PROGRESS -> RESOLVED -> CLOSED

Alternative outcome: REPORTED -> FALSE / DUPLICATE.

11. Data Model

Incident: incident_id, type, severity, confidence, status, created_at, updated_at, location, description, people_affected, injuries, hazards, source

Reporter: reporter_id, anonymous_identifier, relationship_to_incident

Conversation: conversation_id, incident_id, speaker, timestamp, transcript

Event: event_id, incident_id, timestamp, event_type, actor, description

12. Agent Tools

The LLM should use explicit tools rather than directly manipulating the database:

get_current_location() | create_incident() | update_incident() | get_incident() | search_nearby_incidents() | find_duplicate_incidents() | get_emergency_protocol() | notify_operator() | assign_responder() | update_incident_status() | get_incident_statistics()

13. Functional Requirements

ID	Requirement	Description
FR-01	Voice Input	Allow the user to communicate through microphone input.
FR-02	Realtime Transcription	Convert spoken input into text with low latency.
FR-03	Voice Output	Return spoken responses.
FR-04	Interruptions	Allow users to interrupt the agent naturally.
FR-05	Emergency Classification	Classify emergency type.
FR-06	Severity	Assign LOW, MEDIUM, HIGH, or CRITICAL severity.
FR-07	Entity Extraction	Extract relevant emergency entities.

FR-08	Dynamic Questions	Generate follow-up questions based on missing critical information.
FR-09	Incident Creation	Create a structured incident.
FR-10	Location	Support GPS and user-described locations.
FR-11	Incident Updates	Update an incident as information changes.
FR-12	Command Center	Allow operators to view active incidents.
FR-13	Filtering	Filter by severity, type, status, location, and time.
FR-14	Voice Operations	Perform selected dashboard actions through voice.
FR-15	Duplicate Detection	Identify potentially duplicate incidents.
FR-16	Audit Trail	Maintain a chronological event log.

14. Non-Functional Requirements

- Performance: Voice interaction should feel near real-time; UI updates should occur within seconds.
- Reliability: Graceful degradation for microphone, location, network, speech, and AI failures.
- Security: Minimize PII, encrypt data in transit, authenticate command-center access, and audit operator actions.
- Privacy: Do not retain voice recordings unnecessarily; prefer synthetic demo data.

15. Safety Requirements

This is a critical product requirement. The system must clearly communicate that it does not replace emergency services or trained professionals.

- Do not claim to diagnose a patient.
- Do not guarantee safety or outcomes.
- Do not pretend emergency services were contacted when they were not.
- Do not invent a location, responder availability, or action result.
- State clearly when an action is simulated or unavailable.
- Use conservative, reviewed, safety-oriented guidance.

16. MVP Scope

Required: realtime voice, AssemblyAI integration, emergency classification, dynamic questions, incident extraction, severity, location, structured incident creation, database, command center, incident map/list, transcript, interruption handling, mock escalation, and a demo-safe workflow.

Nice to have: multiple-witness fusion, duplicate detection, multilingual support, RAG, voice-controlled command center, responder interface.

17. Recommended Hackathon Demo

1	Report	User says: "There is a major accident on the highway. Two cars crashed and I think someone is trapped."
2	Safety check	Agent asks whether the user is safely away from the vehicles.

3	Dynamic triage	Agent asks how many people are injured and whether anyone is unconscious.
4	Structured incident	System updates a CRITICAL incident with extracted facts and location.
5	Command center	Incident appears immediately in the operator dashboard.
6	Voice operator action	Operator says: "Show me incident 1042" and then "Mark it as responder assigned."
7	Audit	Dashboard shows the updated status and event timeline.

18. Suggested Technology Stack

Frontend: Next.js / React for voice interface, command center, incident map, incident detail, and realtime updates.

Backend: FastAPI (Python) or Node.js for agent orchestration, tool execution, incident management, authentication, APIs, and WebSockets.

Voice: AssemblyAI for realtime speech and voice-agent capabilities.

LLM: Tool-calling model for reasoning, classification, extraction, question generation, and tool selection.

Database: PostgreSQL for incidents, conversations, events, users, responders, and locations.

Realtime: WebSockets or Server-Sent Events.

Maps: Map provider with geolocation and incident markers.

RAG: PostgreSQL pgvector or another vector store for trusted emergency documentation.

19. Engineering Success Metrics

Success Metric	Target
Voice sessions	>= 95% successful sessions
Information extraction	>= 90% accuracy on predefined demo scenarios
Incident creation	>= 95% successful incident creation
Emergency classification	>= 90% correct classification on test scenarios
Operator workflow	>= 90% success for supported voice commands
Perceived latency	Near-real-time conversational experience

20. Key Differentiators

- Voice is the primary interaction model, not an add-on.
- Dynamic questions replace rigid forms.
- Incident state evolves continuously during the conversation.
- Multiple witness reports can contribute to one incident.
- Operators can control selected workflows using voice.
- Agent uses explicit tools for controlled actions.
- Context-aware speech makes the conversation more natural.

21. Roadmap

Phase 1 - Hackathon MVP: Voice reporting -> AssemblyAI -> emergency agent -> classification -> extraction -> incident engine -> command center.

Phase 2: Multilingual voice, duplicate detection, RAG, voice operator controls, responder interface.

Phase 3: Authorized emergency-service integrations, maps, SMS, telephony, IoT sensors, CCTV, wearables.

Phase 4: Multi-agency coordination, predictive incident detection, large-scale emergency operations, and cross-incident intelligence.

22. Hackathon MVP Boundary

Do not attempt to build the entire future roadmap. The strongest submission should demonstrate one polished vertical slice:

User voice -> AssemblyAI -> emergency agent -> classify/extract/question -> incident engine -> command center -> operator action

23. Product Positioning

One-line pitch: An AI-powered emergency response agent that lets people report emergencies naturally by voice while automatically turning chaotic conversations into structured, actionable incidents.

Short pitch: Emergency Response Voice Agent is a real-time AI emergency coordination system powered by AssemblyAI. Users simply describe an emergency by voice, and the agent dynamically gathers critical information, assesses urgency, creates a structured incident, and provides safety-oriented guidance. Operators can monitor incidents through a live command center and control supported workflows using voice.

Tagline: Don't navigate an emergency. Talk through it.

24. Final Product Principle

Listen -> Understand -> Clarify -> Assess -> Structure -> Coordinate -> Act -> Report.

The AI should not simply answer the user. It should turn a chaotic voice conversation into a controlled, auditable, actionable emergency workflow.